

Forms, AJAX Patterns, and Error Handling

This covers form submission, validation, AJAX techniques, and error handling patterns.

1) Form Handling Patterns

1.1 Controlled Components in React

```
function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [errors, setErrors] = useState({});

  const validate = () => {
    const newErrors = {};
    if (!email) newErrors.email = "Email required";
    if (!password) newErrors.password = "Password required";
    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
  };

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!validate()) return;

    try {
      const res = await fetch("/api/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ email, password })
      });

      if (!res.ok) throw new Error("Login failed");
      const data = await res.json();
      // handle success
    } catch (err) {
      setErrors({ form: err.message });
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      {errors.email && <span className="error">{errors.email}</span>}
    </form>
  );
}
```

```

    <input
      value={password}
      type="password"
      onChange={(e) => setPassword(e.target.value)}
      placeholder="Password"
    />
    {errors.password && <span className="error">{errors.password}</span>}

    {errors.form && <div className="error">{errors.form}</div>}

    <button type="submit">Login</button>
  </form>
);
}

```

Interview point:

- controlled components: React state is source of truth.
- update state on every change.

1.2 Uncontrolled Components

```

function FileUpload() {
  const inputRef = useRef(null);

  const handleSubmit = () => {
    const file = inputRef.current.files[0];
    console.log("File:", file.name);
    // use FormData for file upload
  };

  return (
    <>
      <input type="file" ref={inputRef} />
      <button onClick={handleSubmit}>Upload</button>
    </>
  );
}

```

Use case:

- file inputs, since they're uncontrolled by design.

1.3 Form Library: react-hook-form

```

import { useForm } from "react-hook-form";

function LoginForm() {
  const { register, handleSubmit, formState: { errors } } = useForm();

  const onSubmit = async (data) => {

```

```

const res = await fetch("/api/login", {
  method: "POST",
  body: JSON.stringify(data)
});
// handle response
};

return (
  <form onSubmit={handleSubmit(onSubmit)}>
    <input
      {...register("email", {
        required: "Email required",
        pattern: {
          value: /^[^\\w.-]+@[^\\w.-]+\\.\\w+$/,
          message: "Invalid email"
        }
      })}
      placeholder="Email"
    />
    {errors.email && <span>{errors.email.message}</span>}

    <input
      {...register("password", {
        required: "Password required",
        minLength: { value: 8, message: "Min 8 chars" }
      })}
      type="password"
      placeholder="Password"
    />
    {errors.password && <span>{errors.password.message}</span>}

    <button type="submit">Login</button>
  </form>
);
}

```

Advantages:

- less re-renders (uncontrolled by default).
- simpler validation.
- lightweight.

1.4 Formik Library

```

import { Formik, Form, Field, ErrorMessage } from "formik";
import * as Yup from "yup";

const validationSchema = Yup.object().shape({
  email: Yup.string().email("Invalid email").required("Required"),
  password: Yup.string().min(8, "Min 8 chars").required("Required")
});

```

```

function LoginForm() {
  return (
    <Formik
      initialValues={{ email: "", password: "" }}
      validationSchema={validationSchema}
      onSubmit={async (values) => {
        const res = await fetch("/api/login", {
          method: "POST",
          body: JSON.stringify(values)
        });
      }}
    >
      {({ isSubmitting }) => (
        <Form>
          <Field name="email" placeholder="Email" />
          <ErrorMessage name="email" component="span" />

          <Field name="password" type="password" placeholder="Password" />
          <ErrorMessage name="password" component="span" />

          <button type="submit" disabled={isSubmitting}>
            {isSubmitting ? "Logging in..." : "Login"}
          </button>
        </Form>
      )}
    </Formik>
  );
}

```

2) AJAX Patterns: fetch vs axios

2.1 Fetch API

```

// Simple GET
const data = await fetch("/api/users")
  .then(r => r.json());

// POST with error handling
const res = await fetch("/api/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${token}`
  },
  body: JSON.stringify({ name: "Alice" })
});

if (!res.ok) throw new Error(`HTTP ${res.status}`);
const user = await res.json();

```

2.2 Fetch with Abort Controller

```
const controller = new AbortController();

const timeoutId = setTimeout(() => {
  controller.abort(); // cancel request after 5s
}, 5000);

try {
  const res = await fetch("/api/slow", {
    signal: controller.signal
  });
} catch (err) {
  if (err.name === "AbortError") {
    console.log("Request timeout");
  }
} finally {
  clearTimeout(timeoutId);
}
```

2.3 Axios Library

```
import axios from "axios";

// Instance with default config
const api = axios.create({
  baseURL: "https://api.example.com",
  timeout: 5000,
  headers: {
    "Content-Type": "application/json"
  }
});

// Interceptor for auth
api.interceptors.request.use((config) => {
  config.headers.Authorization = `Bearer ${getToken()}`;
  return config;
});

// Interceptor for error handling
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // handle unauthorized
      redirectToLogin();
    }
    return Promise.reject(error);
  }
);
```

```
// Usage
const user = await api.get("/users/1");
await api.post("/users", { name: "Bob" });
```

2.4 Fetch vs Axios Comparison

Fetch:

- built-in, no dependency.
- verbose error handling.
- must manually handle JSON.

Axios:

- automatic JSON transform.
- request/response interceptors.
- timeout, retry, cancel built-in.

```
// Fetch: verbose
const res = await fetch(url, { signal: controller.signal });
if (!res.ok) throw new Error();
const data = await res.json();

// Axios: concise
const { data } = await api.get(url);
```

3) Error Handling Patterns

3.1 Error Boundary for React

```
class ErrorBoundary extends React.Component {
  state = { hasError: false, error: null };

  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  componentDidCatch(error, errorInfo) {
    console.error("Error:", error, errorInfo);
    // send error to logging service
    logErrorToService(error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return (
        <div>
          <h1>Something went wrong</h1>
          <p>{this.state.error?.message}</p>
          <button onClick={() => window.location.reload()}>

```

```

        Reload page
      </button>
    </div>
  );
}
return this.props.children;
}
}

// Use it
<ErrorBoundary>
  <App />
</ErrorBoundary>

```

3.2 Try-Catch with Async/Await

```

async function fetchUserData(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) {
      // treat non-2xx as error
      throw new Error(`HTTP ${res.status}: ${res.statusText}`);
    }
    return await res.json();
  } catch (err) {
    if (err instanceof TypeError) {
      console.error("Network error:", err.message);
    } else if (err instanceof SyntaxError) {
      console.error("Invalid JSON response:", err.message);
    } else {
      console.error("Unexpected error:", err.message);
    }
    throw err; // re-throw for caller to handle
  }
}

```

3.3 Error Handling with Promises

```

fetch("/api/users")
  .then(r => r.json())
  .then(data => console.log(data))
  .catch(err => {
    console.error("Error:", err);
    // show user-friendly message
  })
  .finally(() => {
    // cleanup
    hideSpinner();
  });

```

3.4 Custom Error Classes

```
class APIError extends Error {
  constructor(message, status, data) {
    super(message);
    this.name = "APIError";
    this.status = status;
    this.data = data;
  }
}

class ValidationError extends Error {
  constructor(message, errors) {
    super(message);
    this.name = "ValidationError";
    this.errors = errors;
  }
}

async function submit(formData) {
  try {
    const res = await fetch("/api/submit", {
      method: "POST",
      body: JSON.stringify(formData)
    });

    if (!res.ok) {
      const data = await res.json();
      if (res.status === 400) {
        throw new ValidationError("Validation failed", data.errors);
      } else {
        throw new APIError("Request failed", res.status, data);
      }
    }

    return await res.json();
  } catch (err) {
    if (err instanceof ValidationError) {
      // show field errors
      updateFormErrors(err.errors);
    } else if (err instanceof APIError) {
      // show API error
      showErrorMessage(err.message);
    } else {
      // unknown error
      showErrorMessage("Something went wrong");
    }
    throw err;
  }
}
```


4) File Upload Handling

4.1 FormData for File Upload

```
function FileUpload() {
  const handleChange = async (e) => {
    const file = e.target.files[0];

    const formData = new FormData();
    formData.append("file", file);
    formData.append("description", "User avatar");

    try {
      const res = await fetch("/api/upload", {
        method: "POST",
        body: formData // automatically sets Content-Type: multipart/form-data
      });

      if (!res.ok) throw new Error("Upload failed");
      const result = await res.json();
      console.log("Uploaded:", result.url);
    } catch (err) {
      console.error(err);
    }
  };

  return <input type="file" onChange={handleChange} />;
}
```

4.2 Progress Tracking

```
function FileUploadWithProgress() {
  const [progress, setProgress] = useState(0);

  const handleChange = (e) => {
    const file = e.target.files[0];
    const xhr = new XMLHttpRequest();

    xhr.upload.addEventListener("progress", (ev) => {
      if (ev.lengthComputable) {
        const percent = (ev.loaded / ev.total) * 100;
        setProgress(percent);
      }
    });

    xhr.addEventListener("load", () => {
      console.log("Upload complete");
      setProgress(0);
    });
  };
}
```

```

    const formData = new FormData();
    formData.append("file", file);
    xhr.open("POST", "/api/upload");
    xhr.send(formData);
  };

  return (
    <>
    <input type="file" onChange={handleChange} />
    {progress > 0 && <progress value={progress} max={100} />}
    </>
  );
}

```

5) HTTP Status Codes and Handling

```

const handleResponse = async (res) => {
  if (res.status === 200) {
    // OK
    return res.json();
  } else if (res.status === 201) {
    // Created
    return res.json();
  } else if (res.status === 204) {
    // No Content
    return null;
  } else if (res.status === 400) {
    // Bad Request (validation error)
    const data = await res.json();
    throw new ValidationError(data.message, data.errors);
  } else if (res.status === 401) {
    // Unauthorized (need login)
    redirectToLogin();
    throw new Error("Please login");
  } else if (res.status === 403) {
    // Forbidden (no permission)
    throw new Error("Access denied");
  } else if (res.status === 404) {
    // Not Found
    throw new Error("Resource not found");
  } else if (res.status === 409) {
    // Conflict (duplicate, race condition)
    throw new Error("Request conflicts with current state");
  } else if (res.status === 429) {
    // Too Many Requests (rate limited)
    throw new Error("Rate limited, try again later");
  } else if (res.status >= 500) {
    // Server error
    throw new Error("Server error, try again later");
  }
}

```

```
}  
};
```

6) Revision Checklist

- ☐ Understand controlled vs uncontrolled components.
- ☐ Know react-hook-form and Formik.
- ☐ Know fetch API and abort controller.
- ☐ Know axios and interceptors.
- ☐ Know error boundary pattern.
- ☐ Know custom error classes.
- ☐ Know FormData for file upload.
- ☐ Know HTTP status codes and handling.